Unidad 3

Arrays

Arrays de una y varias dimensiones, cómo crearlos y recorrerlos, la asignación por referencia y los métodos para añadir, quitar, copiar, buscar y ordenar.

Hasta ahora cada variable guardaba un solo valor. Un **array** es una estructura de datos ordenada que almacena varios valores bajo un mismo identificador. En JavaScript los arrays son objetos, su tamaño es elástico (crecen y encogen según hace falta), pueden mezclar tipos de datos y se accede a cada elemento por su **índice**, que empieza en 0.

```
const notas = [7, 4.5, 9, 6];

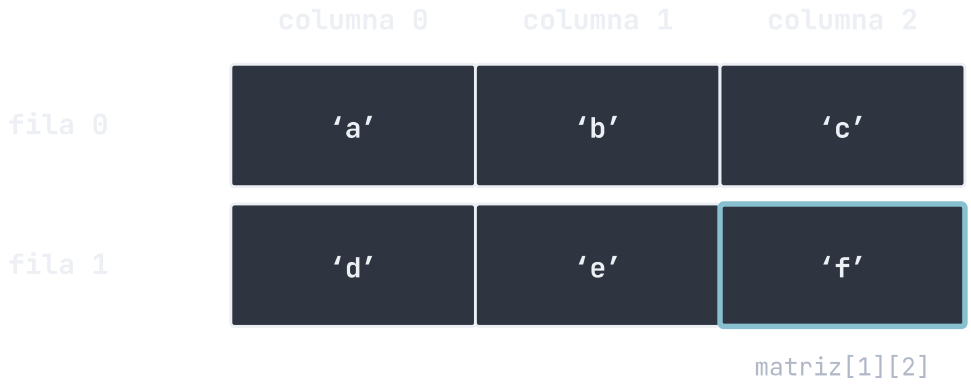
console.log(notas[0]);       // → 7      el primero
console.log(notas[3]);       // → 6      el último
console.log(notas.length);   // → 4      cuántos hay
console.log(notas[4]);       // → undefined, no existe
```

El último elemento siempre está en `notas.length - 1`. Confundir el índice con la posición «humana» (el primero es el 1) es el error más frecuente al empezar con arrays.

Tipos de arrays

- **Unidimensionales**: una lista simple de valores, como `notas` arriba. También se llaman vectores.
- **Bidimensionales**: un array cuyos elementos son a su vez arrays. Se leen como una tabla, con filas y columnas.
- **Multidimensionales**: la idea se puede anidar tantas veces como memoria haya. Un array de tres dimensiones sería como un cubo de Rubik: se accede encadenando índices, `cubo[1][2][0]`.

El diagrama muestra cómo se leen los índices de una matriz de 2 filas y 3 columnas: el primer índice elige la fila y el segundo la columna.



Para llegar a 'f' se elige primero la fila 1 y dentro de ella la columna 2: `matriz[1][2]`. El primer índice siempre corresponde al array exterior.

```
const matriz = [
  ['a', 'b', 'c'],
  ['d', 'e', 'f'],
];

console.log(matriz[1][2]);        // → f
console.log(matriz.length);       // → 2  filas
console.log(matriz[0].length);    // → 3  columnas de la fila 0
```

Creación

Hay tres formas de crear un array, y las tres dan el mismo resultado:

```
const a = new Array();   // constructor
const b = Array();       // sin new, funciona igual
const c = [];            // literal: la forma habitual
```

Con el constructor hay una trampa según cuántos argumentos se pasen:

```
const vacio = new Array(2);      // dos posiciones vacías: [ <2 empty> ]
const pareja = new Array(3, 4);  // dos elementos: [3, 4]
```

Con **un** número crea ese número de posiciones vacías; con varios valores, crea un array con ellos. Por eso se recomienda la notación con corchetes, que no tiene ambigüedad. Con corchetes, una coma sin

valor deja un hueco que se lee como `undefined`, salvo que sea la última coma, que se ignora:

```
const marcas = ['Intel', , 'AMD'];   // hueco en la posición 1
console.log(marcas[1]);              // → undefined
console.log(marcas.length);          // → 3

const dos = ['Intel', 'AMD', ];      // coma final: no crea hueco
console.log(dos.length);             // → 2
```

Recorrido

Hay tres bucles para recorrer un array, y cada uno tiene una pega.

for clásico

Usa un contador desde 0 hasta `length - 1`. Es el que da más control: se sabe el índice, se puede ir hacia atrás o de dos en dos. Su inconveniente es que pasa por las posiciones vacías y las muestra como `undefined`:

```
const marcas = ['Intel', , 'AMD'];

for (let i = 0; i < marcas.length; i++) {
  console.log(i, marcas[i]);
}
// → 0 Intel
// → 1 undefined
// → 2 AMD
```

for … in

Itera sobre los **índices** (como cadenas de texto). Su ventaja es que omite automáticamente las posiciones vacías:

```
for (const i in marcas) {
  console.log(i, marcas[i]);
}
// → 0 Intel
// → 2 AMD
```

for … of

Itera directamente sobre los **valores**, sin variable contador. Es el más cómodo de leer, pero no informa del índice y sí pasa por los huecos:

```
for (const marca of marcas) {
  console.log(marca);
}
// → Intel
// → undefined
// → AMD
```

Bucle	Recorre	Huecos	Índice
`for` clásico	índices numéricos	los muestra como `undefined`	sí
`for … in`	índices como texto	los omite	sí
`for … of`	valores	los muestra como `undefined`	no

Arrays multidimensionales

Se anidan bucles, uno por dimensión. El límite del bucle interior es la longitud de la fila actual, `matriz[i].length`, no un número fijo: así funciona aunque las filas tengan distinto tamaño.

```
const matriz = [
  [1, 2, 3],
  [4, 5, 6],
];

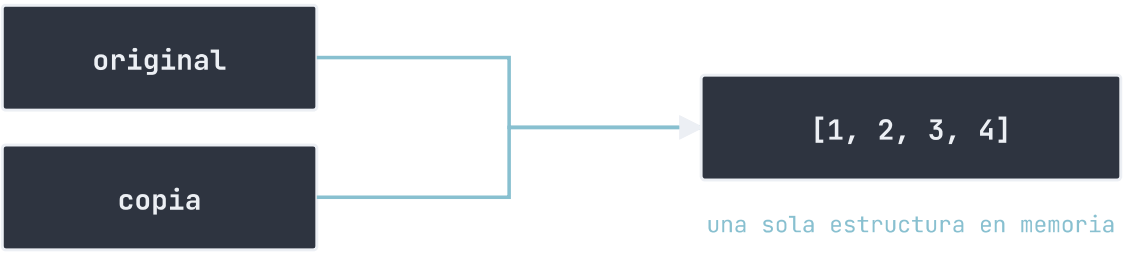
for (let i = 0; i < matriz.length; i++) {
  for (let j = 0; j < matriz[i].length; j++) {
    console.log(`[${i}][${j}] = ${matriz[i][j]}`);
  }
}
```

Asignación: dos nombres, una estructura

Al asignar un array a otra variable **no se copian los datos**. Las dos variables apuntan a la misma estructura en memoria, y modificar una modifica la otra:

```
const original = [1, 2, 3];
const copia = original;       // no es una copia

copia.push(4);
console.log(original);        // → [1, 2, 3, 4]
```



Ambos identificadores son referencias al mismo array. No hay dos listas: hay dos nombres para una.

Para tener una copia independiente hay que crearla explícitamente, por ejemplo con `slice()` (más abajo).

Añadir y eliminar

`push()` añade al final y `unshift()` al principio. Ambos admiten varios valores a la vez y devuelven la nueva longitud del array:

```
const cola = ['b', 'c'];

cola.push('d', 'e');        // → 4   ['b', 'c', 'd', 'e']
cola.unshift('a');          // → 5   ['a', 'b', 'c', 'd', 'e']
```

También se puede asignar directamente a un índice. Si está más allá del final, el array crece y deja huecos hasta llegar:

```
const precios = [10, 20];
```

```
precios[3] = 23.45;          // → [10, 20, <1 empty>, 23.45]
```

Para eliminar, `pop()` quita el último elemento y `shift()` el primero, y los dos devuelven el elemento eliminado:

```
const cola = ['a', 'b', 'c', 'd'];

const ultimo = cola.pop();     // ultimo = 'd'   ['a', 'b', 'c']
const primero = cola.shift();  // primero = 'a'  ['b', 'c']
```

Acortar `length` recorta el array por el final, y `splice(posicion, cantidad)` elimina `cantidad` elementos desde `posicion`, en cualquier punto, y los devuelve en un nuevo array:

```
const letras = ['a', 'b', 'c', 'd', 'e'];

letras.length = 3;                  // ['a', 'b', 'c']

const quitadas = letras.splice(1, 2);
console.log(quitadas);              // → ['b', 'c']
console.log(letras);                // → ['a']
```

▶ **Qué modifica el array y qué no**

Concatenar y copiar

`concat()` une uno o más arrays al final de otro y devuelve un array **nuevo**, sin modificar los originales:

```
const a = [1, 2];
const b = [3, 4];
const c = a.concat(b);      // → [1, 2, 3, 4]; a y b siguen igual
```

`slice(inicio, fin)` copia una parte del array, desde `inicio` hasta `fin` **sin incluirlo**. Sin argumentos copia el array entero, que es la forma habitual de conseguir una copia independiente:

```
const dias = ['lun', 'mar', 'mié', 'jue', 'vie'];

dias.slice(1, 3);          // → ['mar', 'mié']  el índice 3 no entra
dias.slice(3);             // → ['jue', 'vie']  hasta el final
const copia = dias.slice();  // copia completa e independiente
```

Buscar

`indexOf(valor)` devuelve la posición de la primera coincidencia, o `-1` si no la encuentra. Admite un segundo parámetro con el índice desde el que empezar a buscar. `lastIndexOf(valor)` hace lo mismo desde la derecha, e `includes(valor)` responde sólo si está o no:

```
const letras = ['a', 'b', 'a', 'c'];

letras.indexOf('a');       // → 0
letras.indexOf('a', 1);    // → 2   busca a partir del índice 1
letras.lastIndexOf('a');   // → 2
letras.indexOf('z');       // → -1
letras.includes('c');      // → true
```

La comparación es estricta: `[1, 2].indexOf('1')` da `-1` porque `'1'` es una cadena y `1` un número.

Ordenar e invertir

`sort()` ordena el array **modificándolo**, y `reverse()` le da la vuelta. La sorpresa está en el criterio de `sort`: convierte cada elemento a texto y compara según la tabla Unicode. Con palabras en minúscula funciona; con números, mayúsculas o letras como la «ñ» da resultados inesperados:

```
['pera', 'manzana', 'kiwi'].sort();   // → ['kiwi', 'manzana', 'pera']

[10, 9, 100, 1].sort();               // → [1, 10, 100, 9]  ¡como texto!

['ñu', 'oso', 'nutria'].sort();       // → ['nutria', 'oso', 'ñu']

['b', 'a', 'c'].reverse();            // → ['c', 'a', 'b']  sin ordenar
```

`10` queda antes que `9` porque `'1'` va antes que `'9'` en Unicode, y la «ñ» cae detrás de la «o» porque su código está después de todas las letras sin tilde. Para ordenar números hay que pasarle a `sort` una función de comparación, que veremos con las funciones; de momento basta con saber que el orden por defecto no es el numérico.

Ejemplo 1: recorrer una matriz

Escribe la función sumaMatriz(matriz) que reciba un array bidimensional de números y devuelva la suma de todos sus elementos. Recorre las filas con un for y, dentro, las columnas con otro, usando matriz[i].length como límite del bucle interior.

```
function sumaMatriz(matriz) {
  let suma = 0;

  return suma;
}

console.log(sumaMatriz([[1, 2, 3], [4, 5, 6]]));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 1

Ejemplo 2: quitar elementos con splice

Escribe la función quitarDesde(lista, posicion, cantidad) que elimine del array lista los cantidad elementos que empiezan en posicion, usando splice, y devuelva el array de los elementos eliminados. El array original debe quedar modificado.

```
function quitarDesde(lista, posicion, cantidad) {

}

const colores = ['rojo', 'verde', 'azul', 'negro'];
console.log(quitarDesde(colores, 1, 2));
console.log(colores);
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 2

Ejemplo 3: buscar todas las apariciones con indexOf

Escribe la función posiciones(lista, valor) que devuelva un array con todos los índices en los que aparece valor dentro de lista, en orden. Usa indexOf con su segundo parámetro para seguir buscando a partir de la última posición encontrada. Si no aparece, devuelve un array vacío.

```
function posiciones(lista, valor) {
  const encontradas = [];

  return encontradas;
}

console.log(posiciones(['a', 'b', 'a', 'c', 'a'], 'a'));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 3

De este módulo

T3.1 Ejercicios: Arrays
Once programas con arrays unidimensionales y bidimensionales, desde la diagonal de una matriz hasta un mini-buscaminas y un gestor de tareas.
